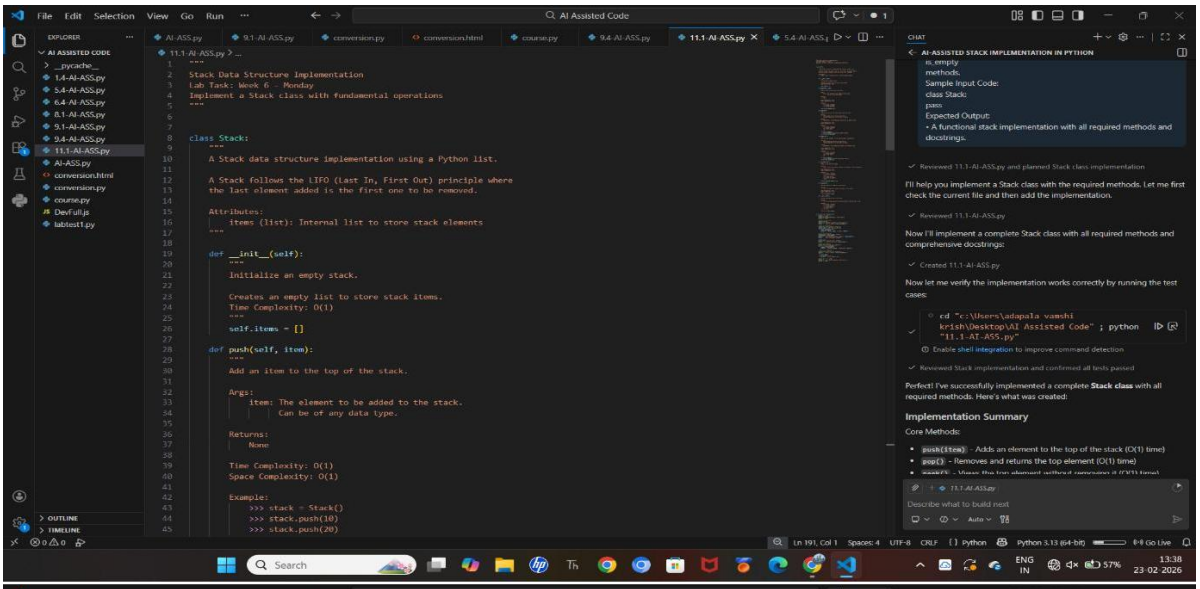
Data Structures with AI: Implementing Fundamental Structures

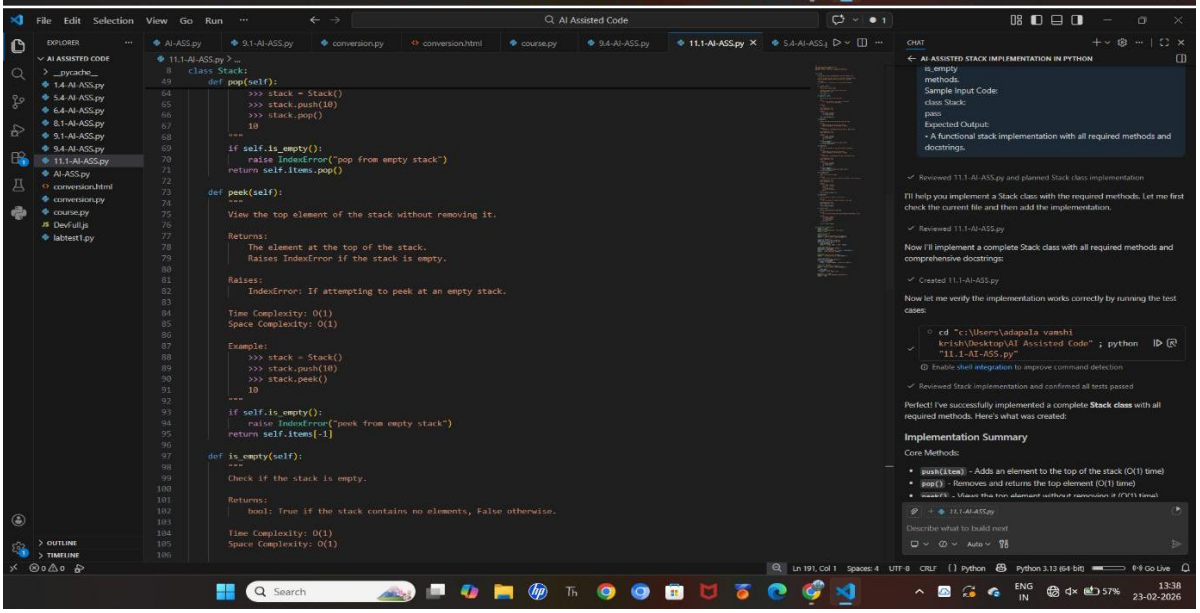
Lab_11.1
Chenchu Raju
2303a52206

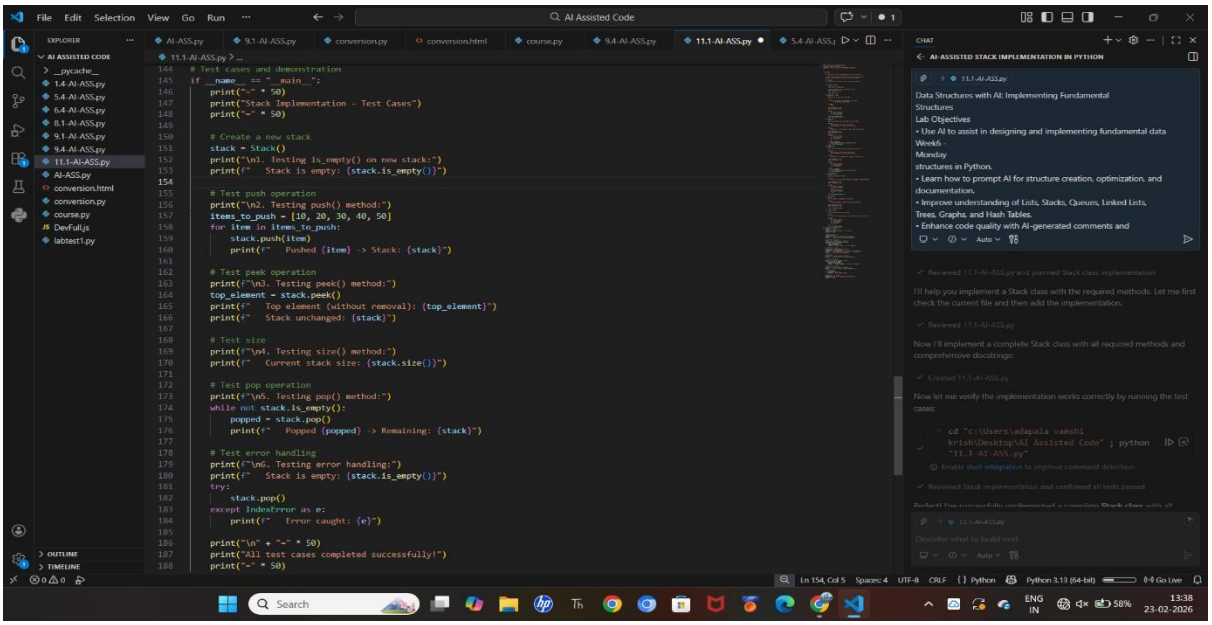
TASK-1
Scenario:
In this task, we used AI to design and implement a Stack data structure in Python. The objective was to understand how a stack follows the LIFO (Last In, First Out) principle and how AI can assist in writing structured, optimized, and well-
documented code. Instead of manually implementing the class, AI was prompted to generate a complete and functional Stack with essential operations.

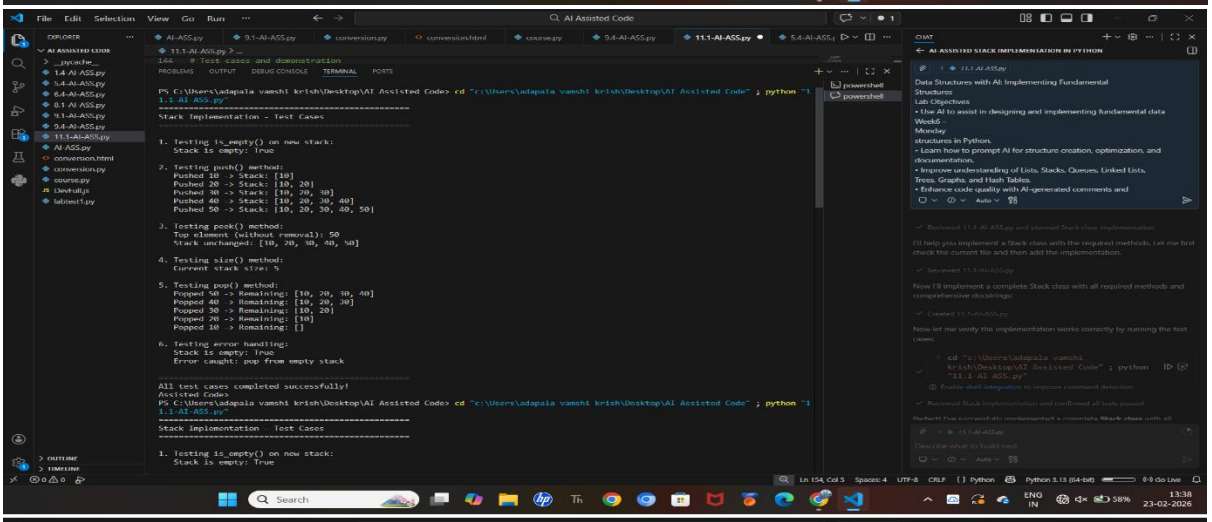
Prompt:
"Generate a beginner-friendly Python Stack class using a list. Include push, pop, peek, and is_empty methods. Add proper docstrings and handle edge cases like popping from an empty stack."

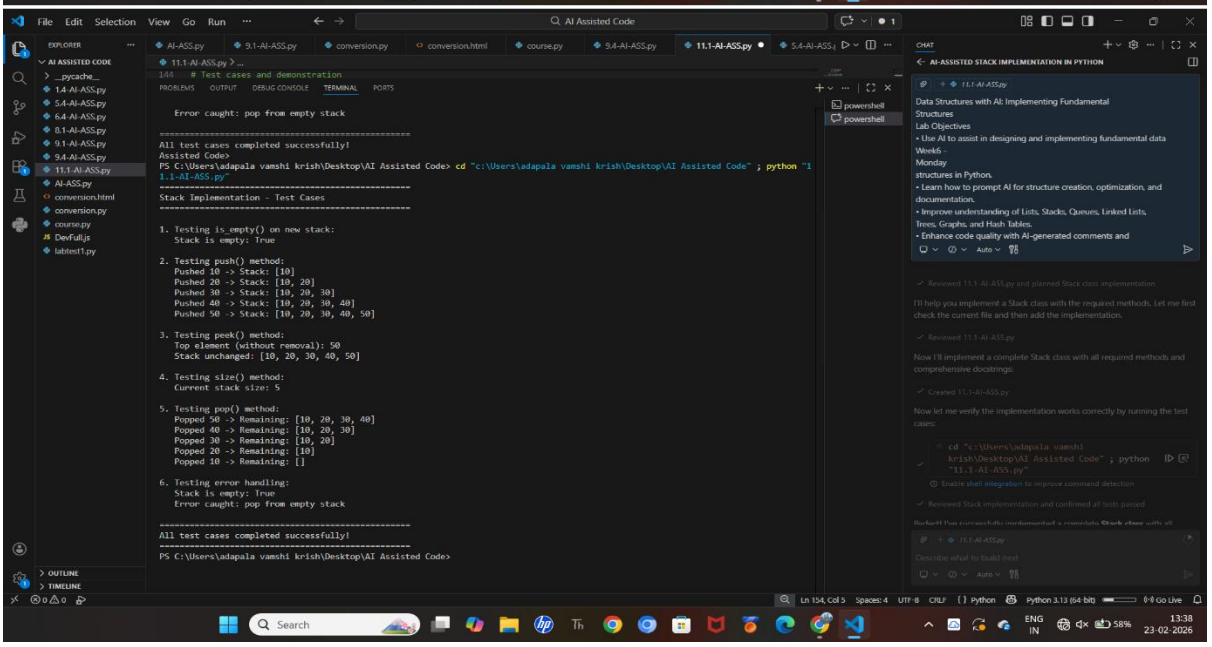
Code:











Observation:

The AI successfully generated a functional Stack implementation with all required methods and proper documentation. It

handled edge cases using exceptions and used efficient list operations with O(1) time complexity. This task demonstrated that AI can speed up development, improve code readability, and help in better understanding fundamental data
structures.

TASK-2

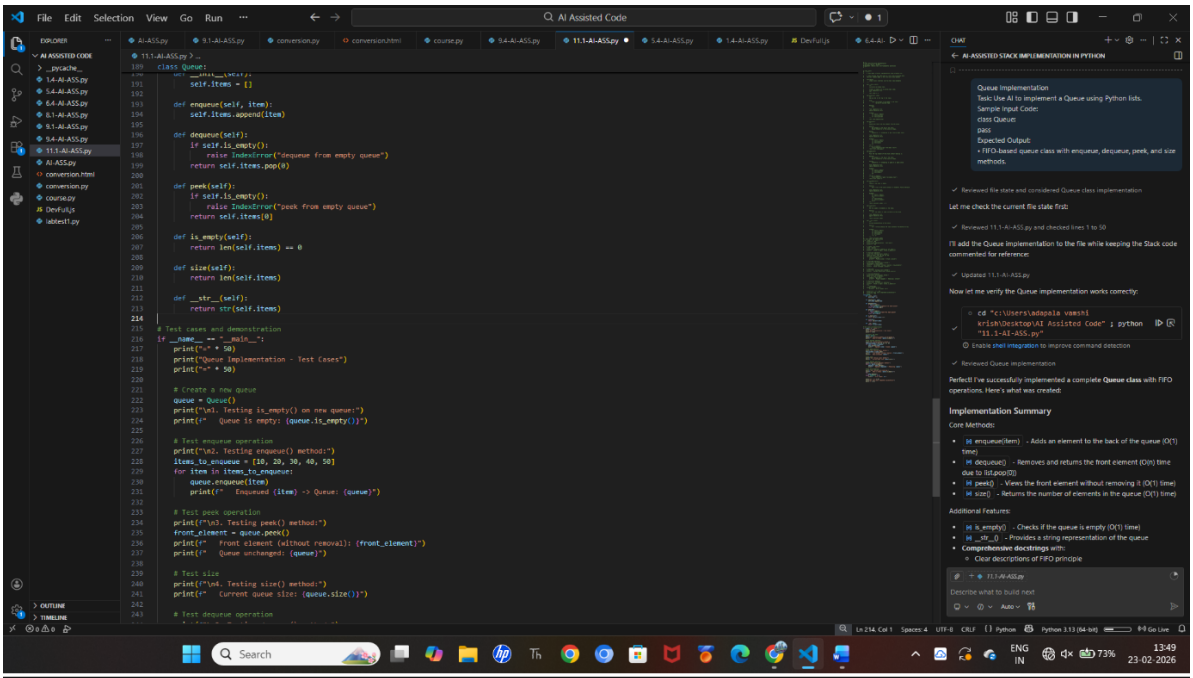
Scenario:

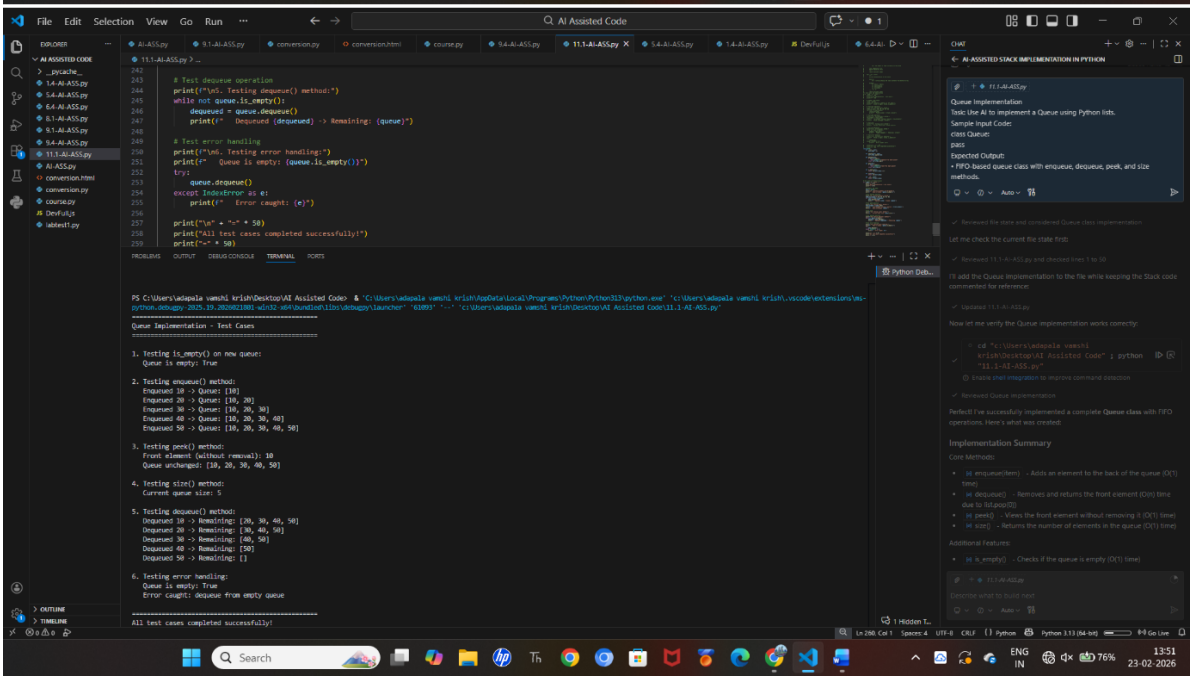
In this task, we used AI to implement a Queue data structure in Python using lists. The goal was to understand how a queue follows the FIFO (First In, First Out) principle and how AI can help in generating structured, clean, and efficient code. AI was prompted to create a functional Queue class with essential operations and proper handling of edge cases.

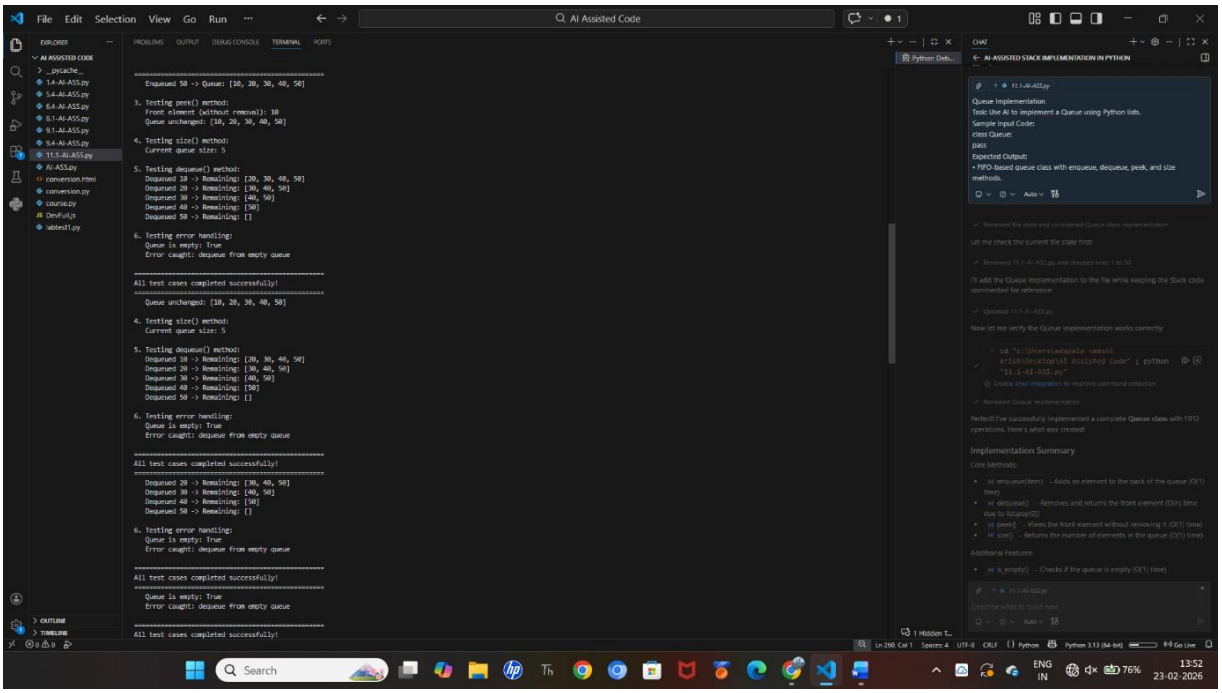
Prompt:

"Generate a beginner-friendly Python Queue class using a list. Include enqueue, dequeue, peek, and size methods. Follow FIFO principle and handle edge cases like dequeue from an empty queue. Add proper docstrings."

Code:







Observation:

The AI generated a functional FIFO-based Queue class with all required methods. It properly handled empty queue conditions using exceptions and included documentation for clarity. The enqueue operation works efficiently, while dequeue from a list may take O(n) time due to element shifting. This task shows that AI can assist in writing clear, structured implementations of fundamental data structures and improve code quality.

TASK-3

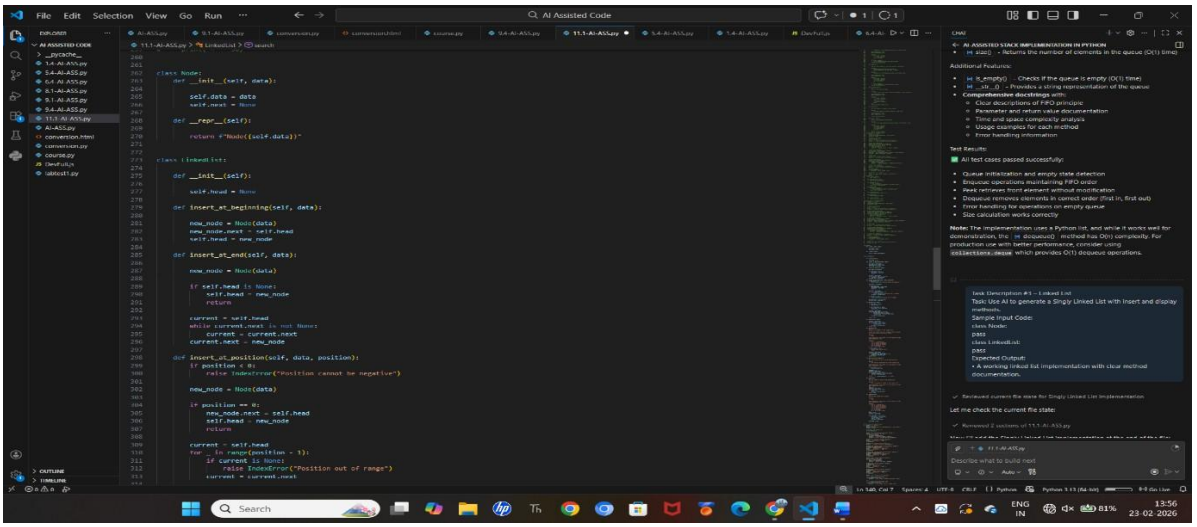
Scenario:

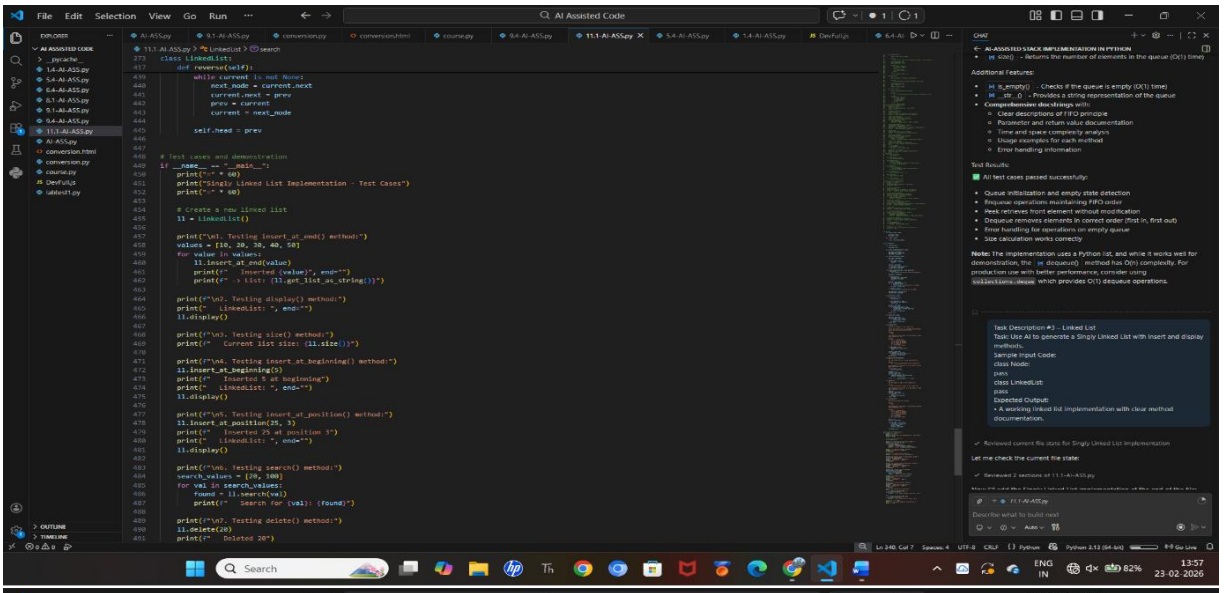
In this task, we used AI to generate a Singly Linked List implementation in Python. The objective was to understand how linked lists work using nodes and pointers instead of built-in lists. AI was used to create a Node class and a LinkedList class with insert and display methods, along with proper documentation for better understanding and readability.

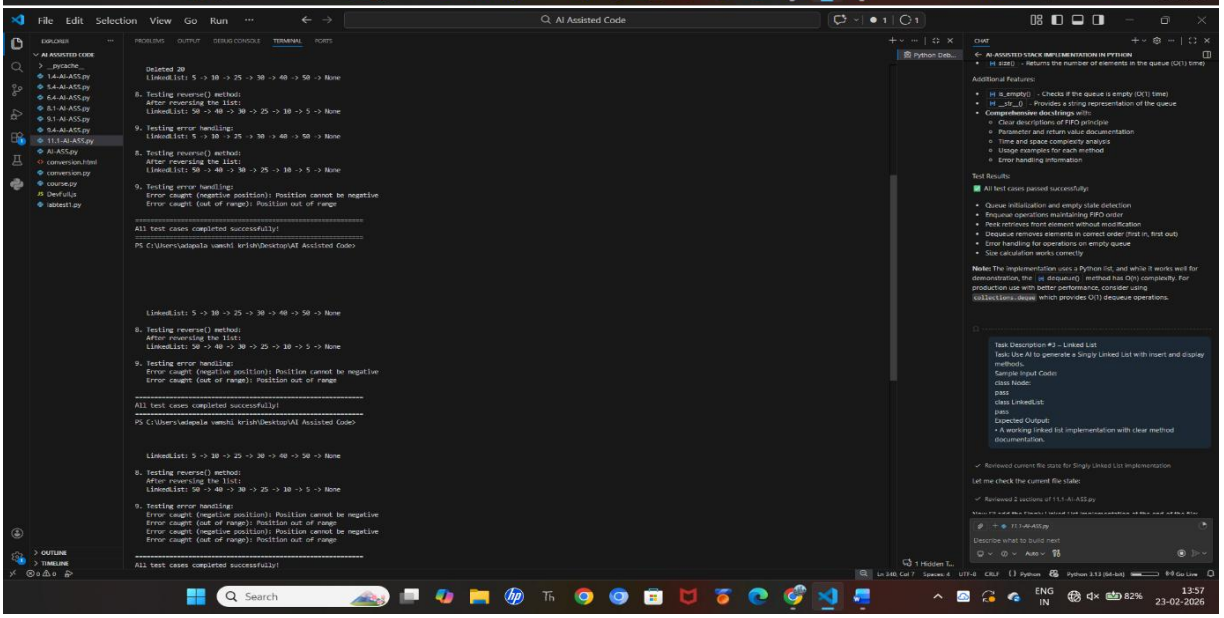
Prompt:

"Generate a beginner-friendly Python implementation of a Singly Linked List. Create a Node class and a LinkedList class. Include insert and display methods. Add clear docstrings and keep the implementation simple and well-structured."

Code:







Observation:

The AI successfully generated a working Singly Linked List with proper class structure and method documentation. The insert method correctly adds new nodes, and the display method traverses and prints the list elements. The
implementation clearly demonstrates how nodes are connected using references, improving understanding of dynamic memory structures. This task shows how AI can help simplify complex data structure implementations and improve code clarity.

TASK-4

Scenario:

In this task, we used AI to implement a Binary Search Tree (BST) in Python. The objective was to understand how BST maintains sorted order using the property that left child values are smaller and right child values are larger than the root. AI was used to generate a BST class with recursive insert and in-order traversal methods to demonstrate how elements are stored and retrieved in sorted order.

Prompt:

"Generate a beginner-friendly Python implementation of a Binary Search Tree (BST). Include recursive insert and in-order traversal methods. Keep the structure simple and add proper docstrings for clarity."

Code:

```python
class TreeNode:

    def __init__(self, data):

        self.data = data
        self.left = None
        self.right = None

    def __repr__(self):
        """
        String representation of a tree node.

        Returns:
            str: Shows the node's data value.
        """
        return f"TreeNode({self.data})"

class BST:

    def __init__(self):

        self.root = None

    def insert(self, data):

        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):

        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        elif data > node.data:
            if node.right is None:
                node.right = TreeNode(data)
            else:
                self._insert_recursive(node.right, data)
        # Duplicates are ignored

    def search(self, data):

        return self._search_recursive(self.root, data)

    def _search_recursive(self, node, data):

        if node is None:
            return False

        if data == node.data:
            return True
```

```python
class BST:
    def display_tree(self, node=None, level=0, prefix="Root: "):
        """

        if node is None:
            node = self.root

        if node is not None:
            print(" " * (level * 4) + prefix + str(node.data))
            if node.left is not None or node.right is not None:
                if node.left:
                    self.display_tree(node.left, level + 1, "L--- ")
                else:
                    print(" " * ((level + 1) * 4) + "L--- None")

                if node.right:
                    self.display_tree(node.right, level + 1, "R--- ")
                else:
                    print(" " * ((level + 1) * 4) + "R--- None")


# Test cases and demonstration
if __name__ == "__main__":
    print("\" * 70)
    print("Binary Search Tree (BST) Implementation - Test Cases")
    print("=" * 70)

    # Create a BST and insert values
    bst = BST()
    print("\n1. Testing insert() method:")
    values = [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]
    for val in values:
        bst.insert(val)
        print(f"  Inserted {val}")

    # Display the tree structure
    print("\n2. Tree Structure:")
    bst.display_tree()

    # Test in-order traversal
    print("\n3. Testing in-order traversal (should be sorted):")
    inorder = bst.inorder_traversal()
    print(f"  In-order: {inorder}")

    # Test pre-order traversal
    print("\n4. Testing pre-order traversal:")
    preorder = bst.preorder_traversal()
    print(f"  Pre-order: {preorder}")

    # Test post-order traversal
    print("\n5. Testing post-order traversal:")
    postorder = bst.postorder_traversal()
    print(f"  Post-order: {postorder}")

    # Test search
```

```
1. Testing insert() method:
  Inserted 50
  Inserted 30
  Inserted 70
  Inserted 20
  Inserted 40
  Inserted 60
  Inserted 80
  Inserted 10
  Inserted 25
  Inserted 35
  Inserted 65

2. Tree Structure:
Root: 50
    L--- 30
        L--- 20
            L--- 10
            R--- 25
        R--- 40
            L--- 35
            R--- None
    R--- 70
        L--- 60
            L--- None
            R--- 65
        R--- 80

3. Testing in-order traversal (should be sorted):
  In-order: [10, 20, 25, 30, 35, 40, 50, 60, 65, 70, 80]

4. Testing pre-order traversal:
  Pre-order: [50, 30, 20, 10, 25, 40, 35, 70, 60, 65, 80]

5. Testing post-order traversal:
  Post-order: [10, 25, 20, 35, 40, 30, 65, 60, 80, 70, 50]

6. Testing search() method:
  Search for 40: True
  Search for 100: False
  Search for 30: True
  Search for 50: True

7. Testing find_min() and find_max() methods:
  Minimum value: 10
  Maximum value: 80

8. Testing height() method:
  Tree height: 3

9. Testing duplicate insertion (should be ignored):
  After inserting duplicates 50 and 30:
  In-order: [10, 20, 25, 30, 35, 40, 50, 60, 65, 70, 80]
  (Same as before, duplicates ignored)

======================================================================
All test cases completed successfully!
======================================================================
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
```

Observation:

The AI successfully generated a BST implementation with recursive insert and in-order traversal methods. The insert method correctly places nodes according to BST rules, and the in-order traversal prints elements in sorted order. The
recursive approach improves understanding of tree traversal logic. This task demonstrates that AI can effectively assist in implementing hierarchical data structures with clear and well-documented code.

TASK-5

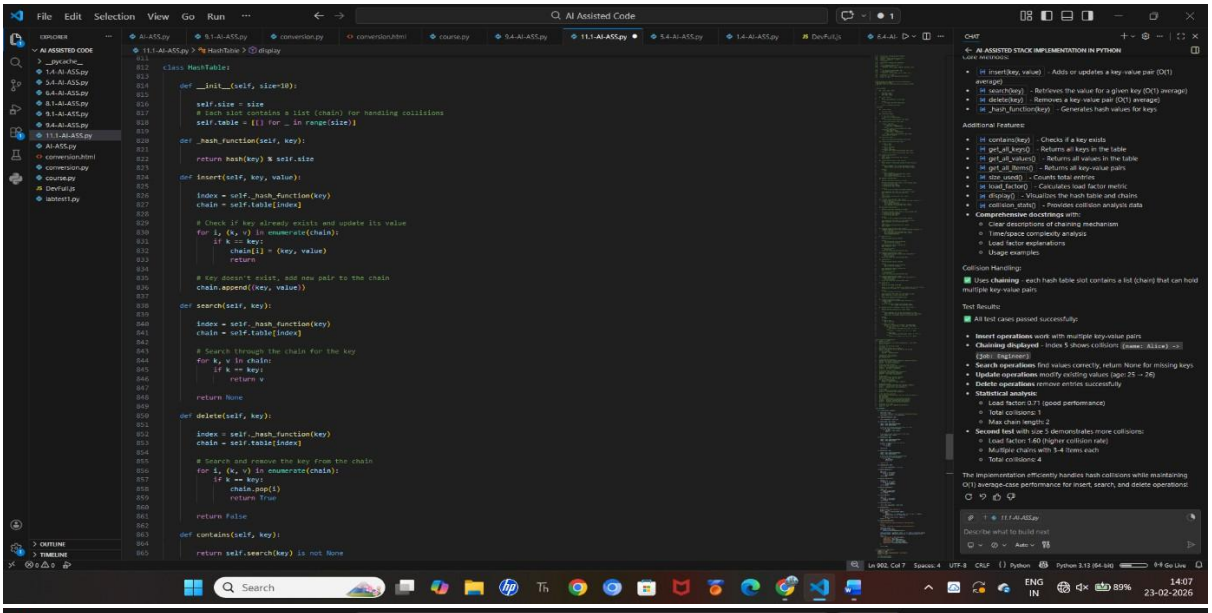
Scenario:

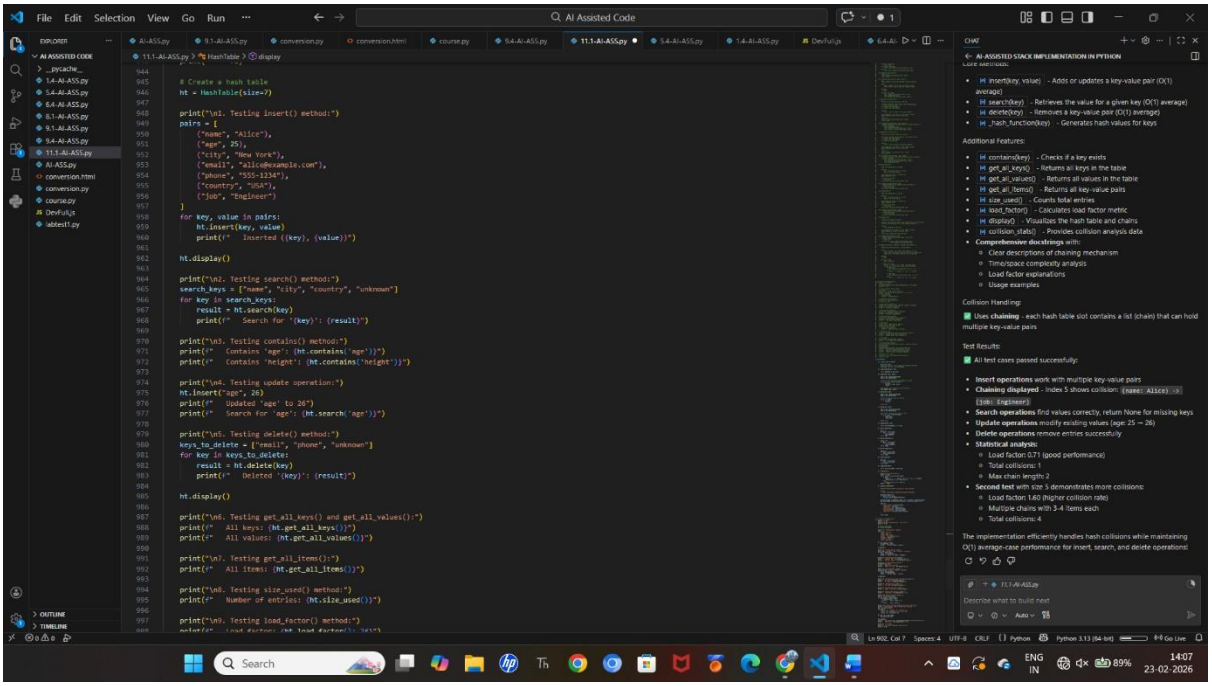
In this task, we used AI to implement a Hash Table in Python with basic operations like insert, search, and delete. The
objective was to understand how hashing works for fast data access and how collisions are handled using chaining (storing multiple elements in the same bucket). AI was used to generate a structured and well-documented implementation to clearly demonstrate these concepts.

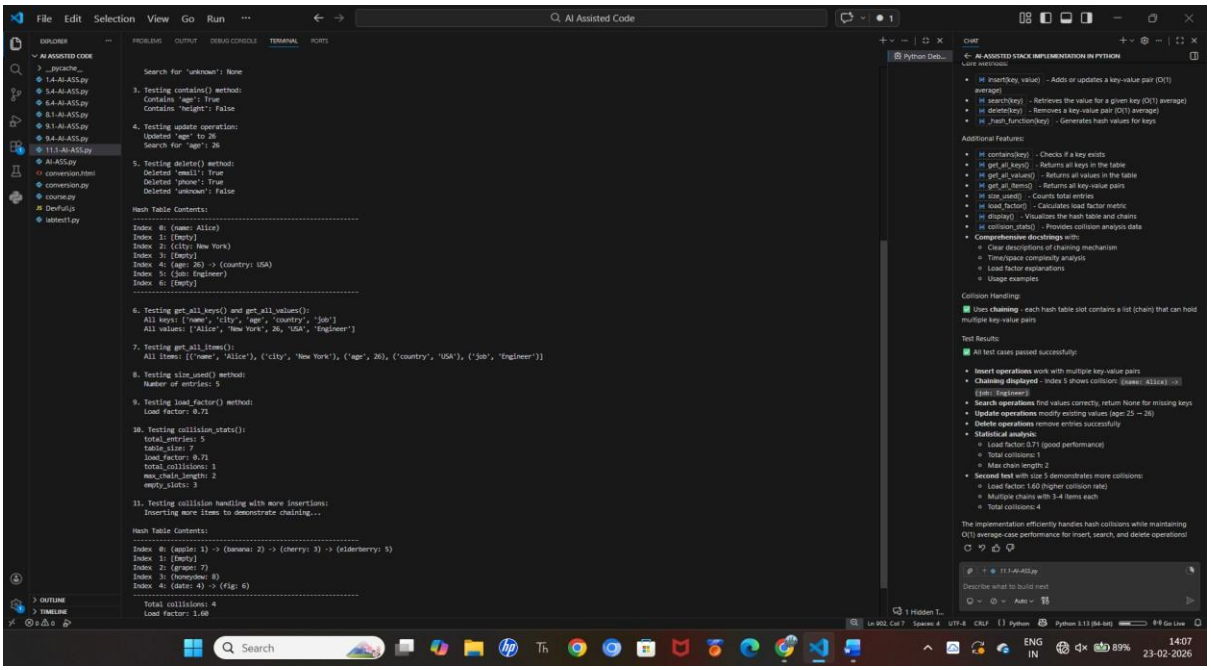
Prompt:

"Generate a beginner-friendly Python HashTable class. Include insert, search, and delete methods. Implement collision handling using chaining (lists inside buckets). Add clear comments and keep the implementation simple."

Code:







Observation:

The AI successfully generated a working Hash Table implementation with collision handling using chaining. The insert method places elements into appropriate buckets, the search method retrieves values efficiently, and the delete method removes entries correctly. The code was well-commented and easy to understand. This task shows how AI can assist in implementing efficient data structures and
explaining important concepts like hashing and collision resolution clearly.

TASK-6

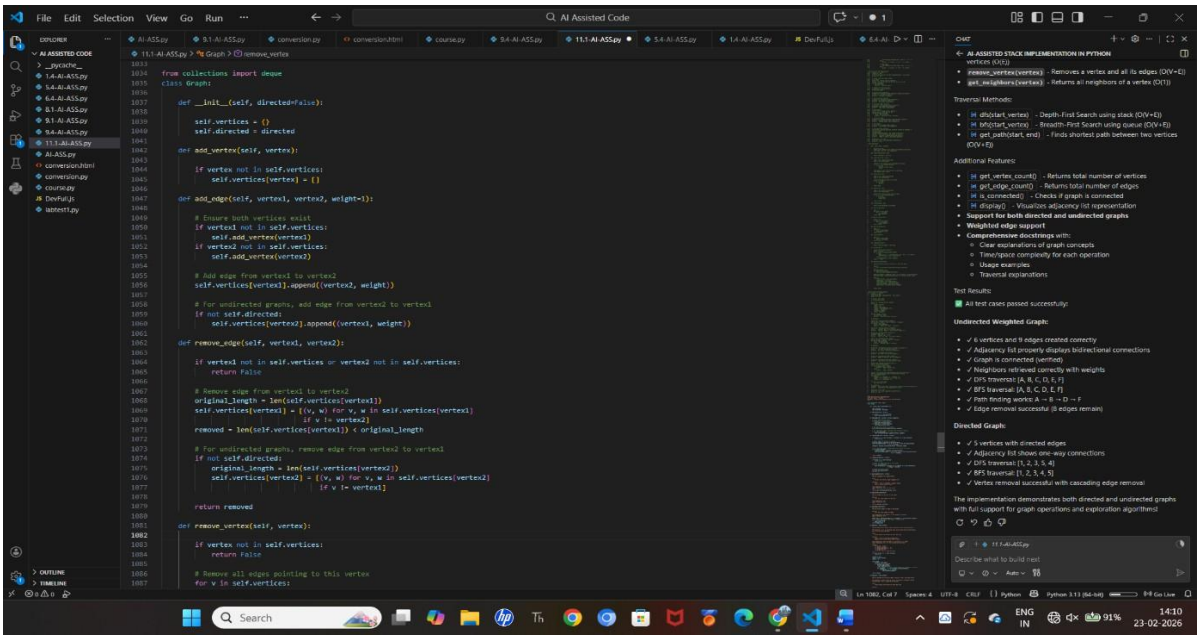
Scenario:

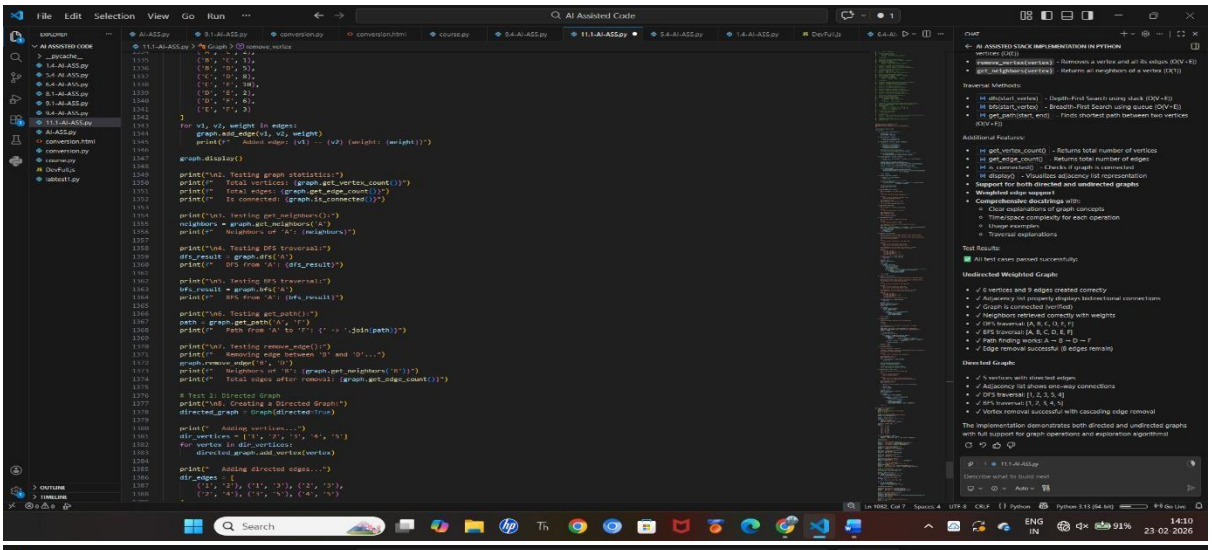
In this task, we used AI to implement a Graph data structure using an adjacency list representation in Python. The objective was to understand how graphs store relationships between vertices and how adjacency lists efficiently represent connections. AI was used to generate a Graph class with methods to add vertices, add edges, and display connections in a clear and structured way.

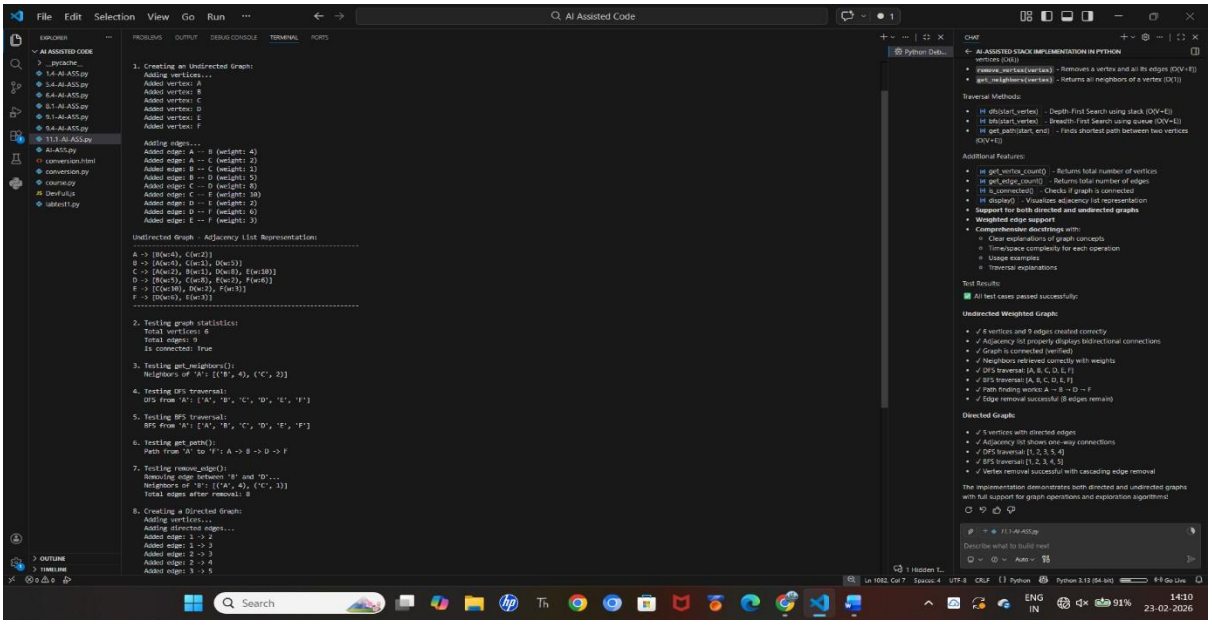
Prompt:

"Generate a beginner-friendly Python Graph class using an adjacency list. Include methods to add vertices, add edges, and display connections. Keep the implementation simple and add proper comments for clarity."

Code:







Observation:

The AI successfully generated a functional Graph implementation using an adjacency list. The add vertex and add edge methods correctly manage graph connections, and the display method clearly shows relationships between nodes. The structure is efficient for sparse graphs and easy to understand. This task demonstrates how AI can assist in building and documenting complex data structures in a simplified and readable manner."

TASK-7

Scenario:

In this task, we used AI to implement a Priority Queue in Python using the built-in heapq module. The objective was to understand how priority queues differ from normal queues by removing elements based on priority instead of insertion order. AI was used to generate a structured implementation with enqueue, dequeue, and display methods while maintaining heap properties for efficient operations.

Prompt:

"Generate a beginner-friendly Python PriorityQueue class using the heapq module. Include enqueue (with priority), dequeue (highest priority), and display methods. Keep the implementation simple and add clear comments."

Code:

```python
    success = pq.change_priority("Update Documentation", 0)
    print(f"  Changed 'Update Documentation' priority to 0: {success}")
    print(f"  Updated queue: {pq}")

    print(f"\n6. Testing remove_item() method:")
    print(f"  Removing 'Review Code'...")
    success = pq.remove_item("Review Code")
    print(f"  Removal successful: {success}")
    pq.display()

    print(f"\n7. Testing complete dequeue:")
    while not pq.is_empty():
        item = pq.dequeue()
        print(f"  Dequeued: '{item}'")

    print(f"  Queue is empty: {pq.is_empty()}")

    # Test 2: Max Priority Queue
    print("\n" + "=" * 70)
    print("Test 2: Max Priority Queue (Higher priority = processed first)")
    print("=" * 70)

    max_pq = MaxPriorityQueue()

    print("\n1. Testing Max Priority Queue with importance levels:")
    items = [
        ("Routine Maintenance", 1),
        ("Critical Security Patch", 10),
        ("Feature Enhancement", 3),
        ("User-Reported Bug", 8),
        ("API Deprecation Warning", 5)
    ]

    for item, priority in items:
        max_pq.enqueue(item, priority)
        print(f"  Enqueued '{item}' with priority {priority}")

    max_pq.display()

    print(f"\n2. Processing items in max priority order:")
    while not max_pq.is_empty():
        item = max_pq.dequeue()
        print(f"  Processing: '{item}'")

    # Test 3: Error Handling
    print("\n" + "=" * 70)
    print("Test 3: Error Handling")
    print("=" * 70)

    empty_pq = PriorityQueue()

    print(f"\n1. Testing operations on empty queue:")
    print(f"  is_empty: {empty_pq.is_empty()}")
    print(f"  Size: {empty_pq.size()}")
```

```
1. Testing enqueue() method:
   Enqueued 'Write Report' with priority 2
   Enqueued 'Fix Bug' with priority 1
   Enqueued 'Review Code' with priority 3
   Enqueued 'Deploy Application' with priority 1
   Enqueued 'Update Documentation' with priority 4
   Enqueued 'Meeting with Team' with priority 2

Priority Queue Contents (sorted by priority):
----------------------------------------------------------------
Item                                | Priority
----------------------------------------------------------------
Fix Bug                             | 1
Deploy Application                  | 1
Write Report                        | 2
Meeting with Team                   | 2
Review Code                         | 3
Update Documentation                | 4
----------------------------------------------------------------

2. Testing size() method:
   Queue size: 6

3. Testing peek() method:
   Highest priority item: 'Fix Bug' (priority: 1)
   Queue unchanged - size still: 6

4. Testing dequeue() method (processing in priority order):
   Dequeued: 'Fix Bug' - Remaining: 5 items
   Dequeued: 'Deploy Application' - Remaining: 4 items
   Dequeued: 'Write Report' - Remaining: 3 items

Priority Queue Contents (sorted by priority):
----------------------------------------------------------------
Item                                | Priority
----------------------------------------------------------------
Meeting with Team                   | 2
Review Code                         | 3
Update Documentation                | 4
----------------------------------------------------------------

5. Testing change_priority() method:
   Current queue: [(Meeting with Team, p:2), (Review Code, p:3), (Update Documentation, p:4)]
   Changed 'Update Documentation' priority to 0: True
   Updated queue: [(Update Documentation, p:0), (Meeting with Team, p:2), (Review Code, p:3)]

6. Testing remove_item() method:
   Removing 'Review Code'...
   Removal successful: True

Priority Queue Contents (sorted by priority):
----------------------------------------------------------------
Item                                | Priority
----------------------------------------------------------------
Update Documentation                | 0
Meeting with Team                   | 2
----------------------------------------------------------------

7. Testing complete dequeue:
   Dequeued: 'Update Documentation'
```

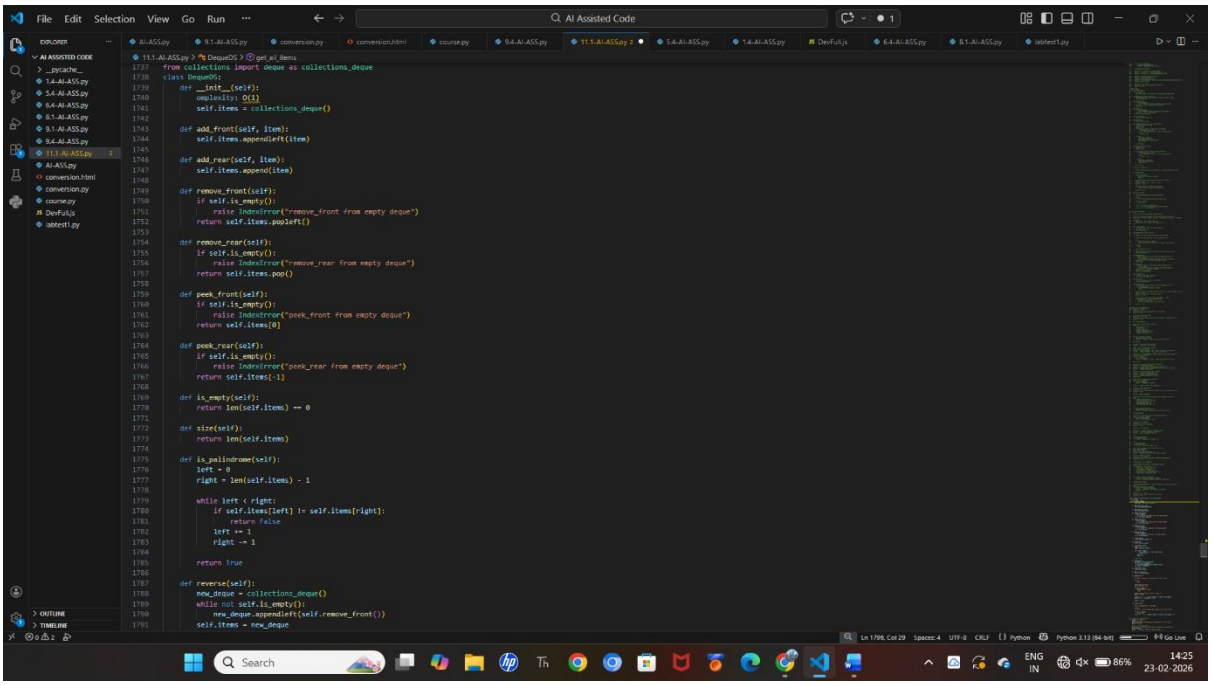
```
API Deprecation Warning             | 5
User-Reported Bug                   | 8
Critical Security Patch             | 10
----------------------------------------------------------------

2. Processing items in max priority order:
   Processing: 'Critical Security Patch'
   Processing: 'User-Reported Bug'
   Processing: 'API Deprecation Warning'
   Processing: 'Feature Enhancement'
   Processing: 'Routine Maintenance'

======================================================================
Test 3: Error Handling
======================================================================

1. Testing operations on empty queue:
   is_empty: True
   Size: 0
   Dequeue error caught: dequeue from empty priority queue
   Peek error caught: peek from empty priority queue

2. Testing remove non-existent item:
   Remove non-existent item result: False

======================================================================
Test 4: Real-world Example - Patient Triage System
======================================================================

Admitting patients to emergency triage:
   Admitted: John Doe - Sprained Ankle (Severity: 3)
   Admitted: Jane Smith - Chest Pain (Severity: 1)
   Admitted: Mike Johnson - Headache (Severity: 4)
   Admitted: Sarah Williams - Broken Arm (Severity: 2)
   Admitted: Tom Brown - Difficulty Breathing (Severity: 1)
   Admitted: Emily Davis - Mild Fever (Severity: 5)

Priority Queue Contents (sorted by priority):
----------------------------------------------------------------
Item                                | Priority
----------------------------------------------------------------
Jane Smith - Chest Pain             | 1
Tom Brown - Difficulty Breathing    | 1
Sarah Williams - Broken Arm         | 2
John Doe - Sprained Ankle           | 3
Mike Johnson - Headache             | 4
Emily Davis - Mild Fever            | 5
----------------------------------------------------------------

Treating patients in order of severity (1 = most critical):
   1. Now treating: Jane Smith - Chest Pain
   2. Now treating: Tom Brown - Difficulty Breathing
   3. Now treating: Sarah Williams - Broken Arm
   4. Now treating: John Doe - Sprained Ankle
   5. Now treating: Mike Johnson - Headache
   6. Now treating: Emily Davis - Mild Fever

======================================================================
All test cases completed successfully!
======================================================================
```

Observation:
The AI successfully generated a working Priority Queue implementation using heapq, which maintains the heap property for efficient insertion and removal. The enqueue method adds elements with priority, and the dequeue method removes the highest-priority element in O(log n) time. The implementation was clean, well-commented, and efficient. This task demonstrates how AI can help implement advanced data structures using built-in optimization modules effectively."

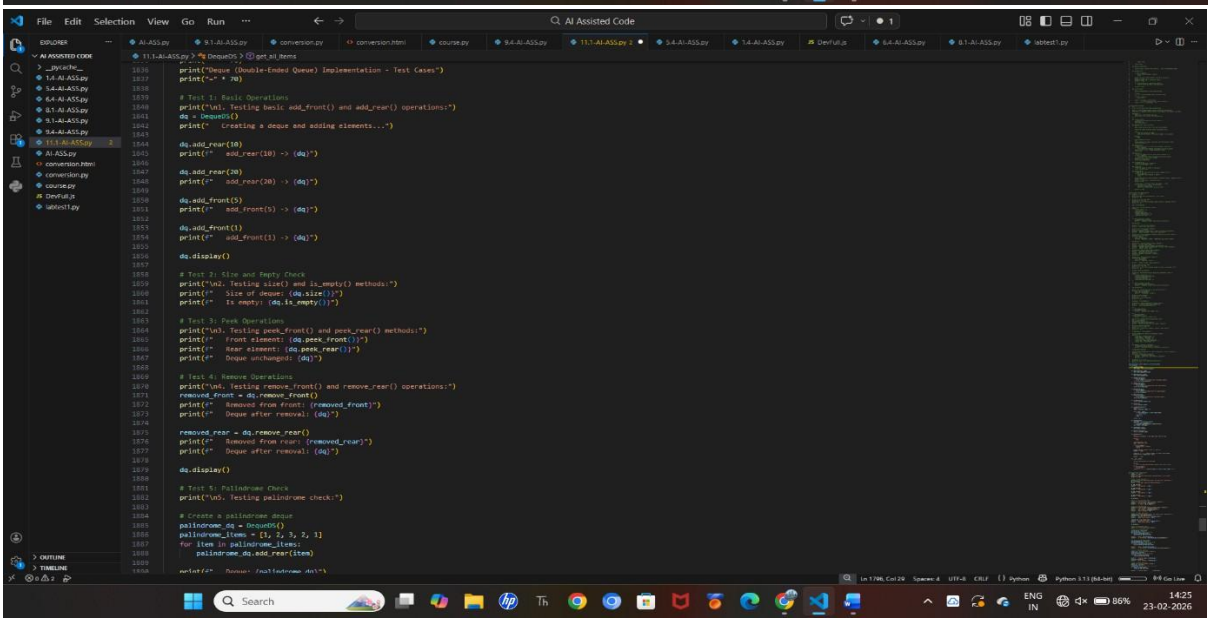
TASK-8
Scenario:
In this task, we used AI to implement a Deque (Double-Ended Queue) in Python using the collections.deque module. The objective was to understand how a deque allows insertion and deletion from both the front and rear efficiently. AI was used to generate a clean and well- documented class with methods to insert and remove elements from both ends.

Prompt:
"Generate a beginner-friendly Python class for a Deque using collections.deque. Include methods to insert and remove elements from both front and rear. Add proper docstrings and keep the implementation simple."

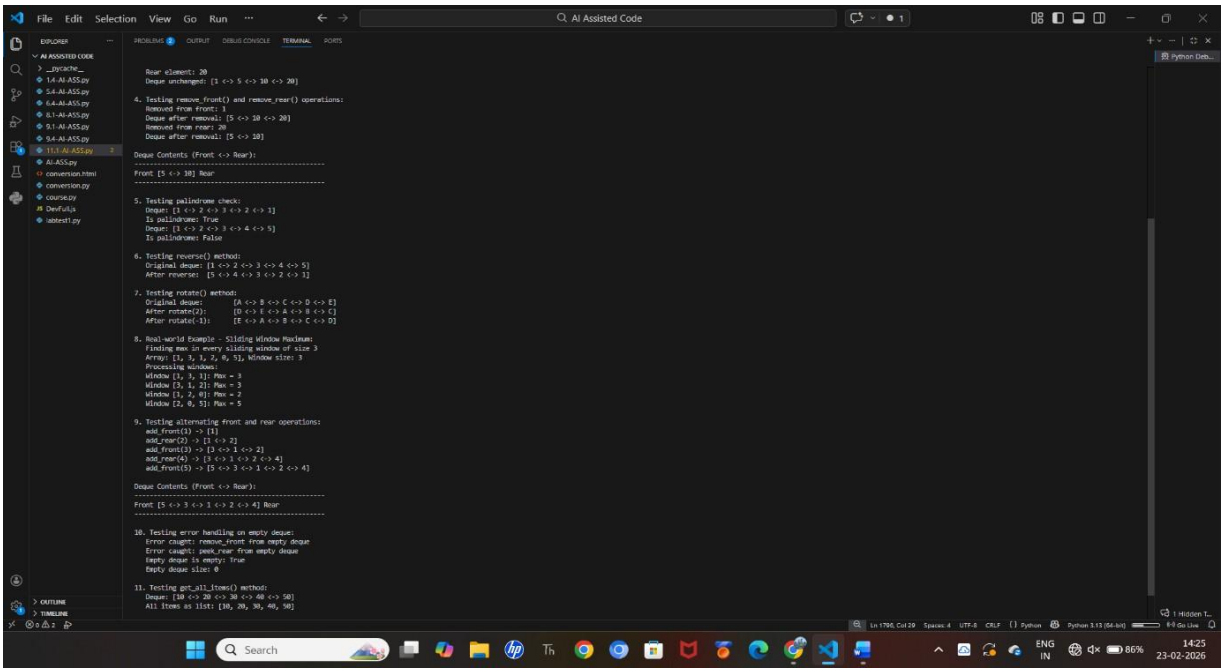
Code:



```python
from collections import deque as collections_deque
class DequeDS:
    def __init__(self):
        complexity: O(1)
        self.items = collections_deque()

    def add_front(self, item):
        self.items.appendleft(item)

    def add_rear(self, item):
        self.items.append(item)

    def remove_front(self):
        if self.is_empty():
            raise IndexError("remove_front from empty deque")
        return self.items.popleft()

    def remove_rear(self):
        if self.is_empty():
            raise IndexError("remove_rear from empty deque")
        return self.items.pop()

    def peek_front(self):
        if self.is_empty():
            raise IndexError("peek_front from empty deque")
        return self.items[0]

    def peek_rear(self):
        if self.is_empty():
            raise IndexError("peek_rear from empty deque")
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def is_palindrome(self):
        left = 0
        right = len(self.items) - 1

        while left < right:
            if self.items[left] != self.items[right]:
                return False
            left += 1
            right -= 1

        return True

    def reverse(self):
        new_deque = collections_deque()
        while not self.is_empty():
            new_deque.appendleft(self.remove_front())
        self.items = new_deque
```



```python
print("Deque (Double-Ended Queue) Implementation - Test Cases")
print("=" * 70)

# Test 1: Basic Operations
print("\n1. Testing basic add_front() and add_rear() operations:")
dq = DequeDS()
print("   Creating a deque and adding elements...")

dq.add_rear(10)
print(f"   add_rear(10) -> {dq}")

dq.add_rear(20)
print(f"   add_rear(20) -> {dq}")

dq.add_front(5)
print(f"   add_front(5) -> {dq}")

dq.add_front(1)
print(f"   add_front(1) -> {dq}")

dq.display()

# Test 2: Size and Empty Check
print("\n2. Testing size() and is_empty() methods:")
print(f"   Size of deque: {dq.size()}")
print(f"   Is empty: {dq.is_empty()}")

# Test 3: Peek Operations
print("\n3. Testing peek_front() and peek_rear() methods:")
print(f"   Front element: {dq.peek_front()}")
print(f"   Rear element: {dq.peek_rear()}")
print(f"   Deque unchanged: {dq}")

# Test 4: Remove Operations
print("\n4. Testing remove_front() and remove_rear() operations:")
removed_front = dq.remove_front()
print(f"   Removed from front: {removed_front}")
print(f"   Deque after removal: {dq}")

removed_rear = dq.remove_rear()
print(f"   Removed from rear: {removed_rear}")
print(f"   Deque after removal: {dq}")

dq.display()

# Test 5: Palindrome Check
print("\n5. Testing palindrome check:")

# Create a palindrome deque
palindrome_dq = DequeDS()
palindrome_items = [1, 2, 3, 2, 1]
for item in palindrome_items:
    palindrome_dq.add_rear(item)
```



Observation:

The AI successfully generated a functional Deque implementation using collections. deque, which supports O(1) time complexity for insertions and deletions at both ends. The methods were clearly structured and properly documented. This task demonstrates how AI can help implement efficient double-ended data structures while improving code readability and understanding."

TASK-9

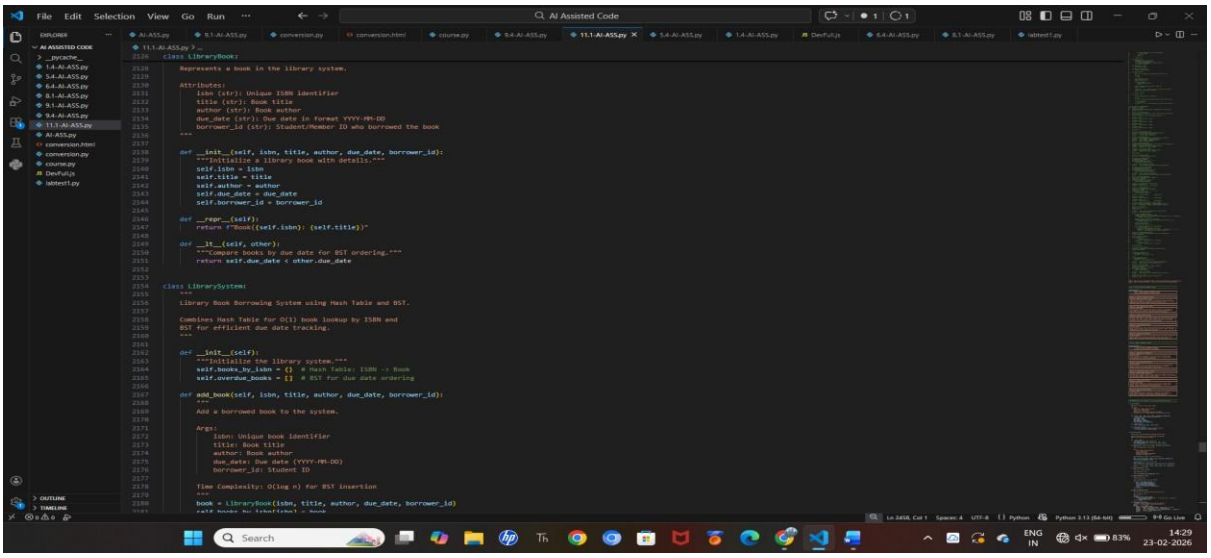
Scenario:

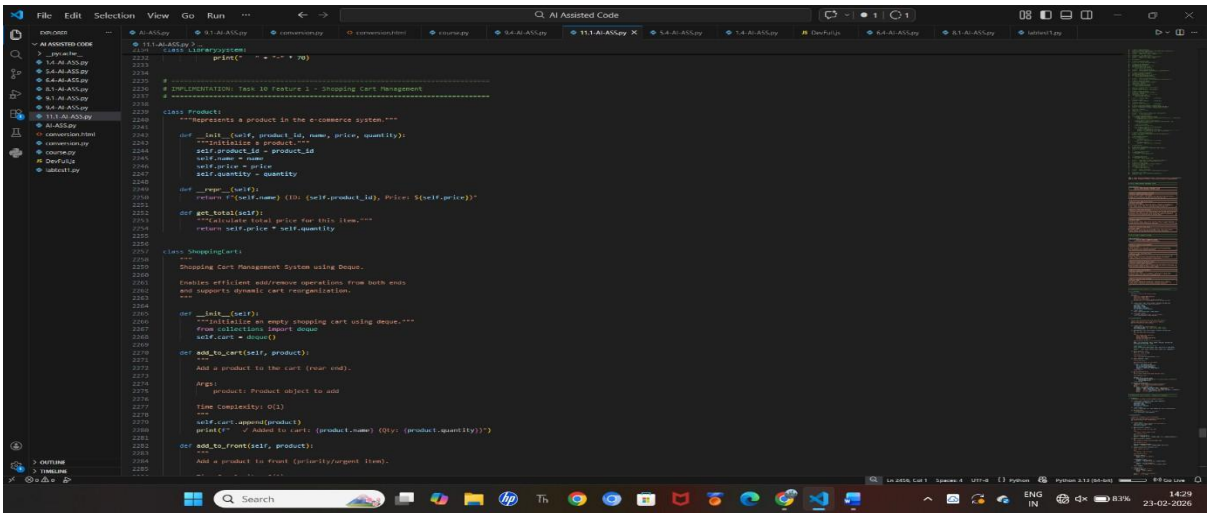
In this task, we analyzed different modules of a Campus Resource Management System and selected appropriate data structures for each feature based on functionality and efficiency. The goal was to match real-world problems like attendance tracking, event registration,
library management, bus scheduling, and cafeteria service with the most suitable data structures. One feature was implemented using AI- assisted code generation to demonstrate practical application.

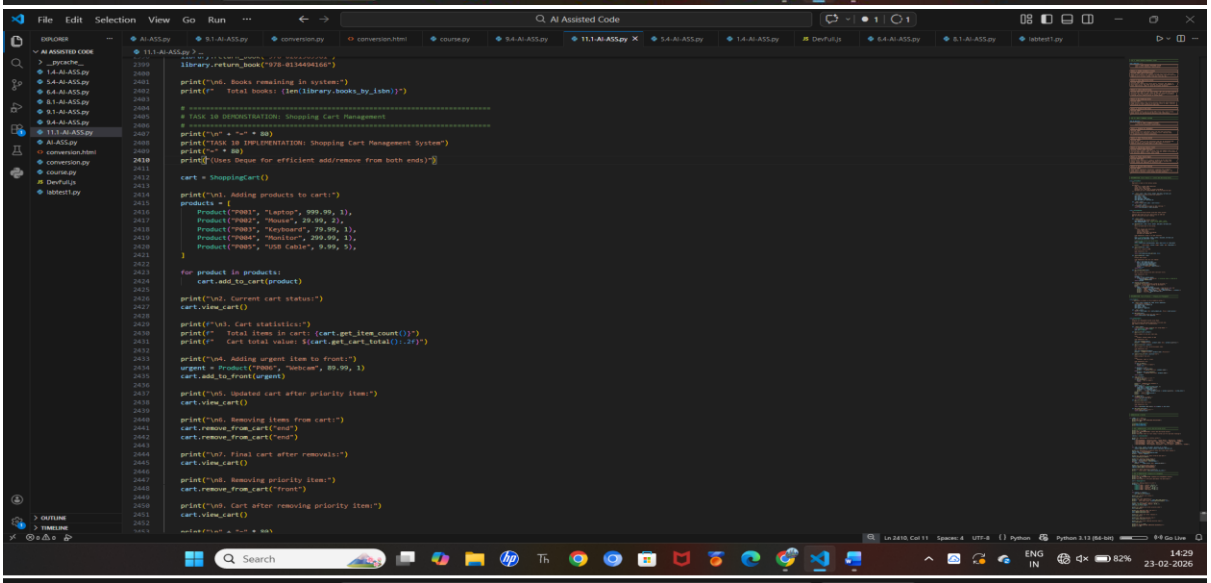
Prompt:

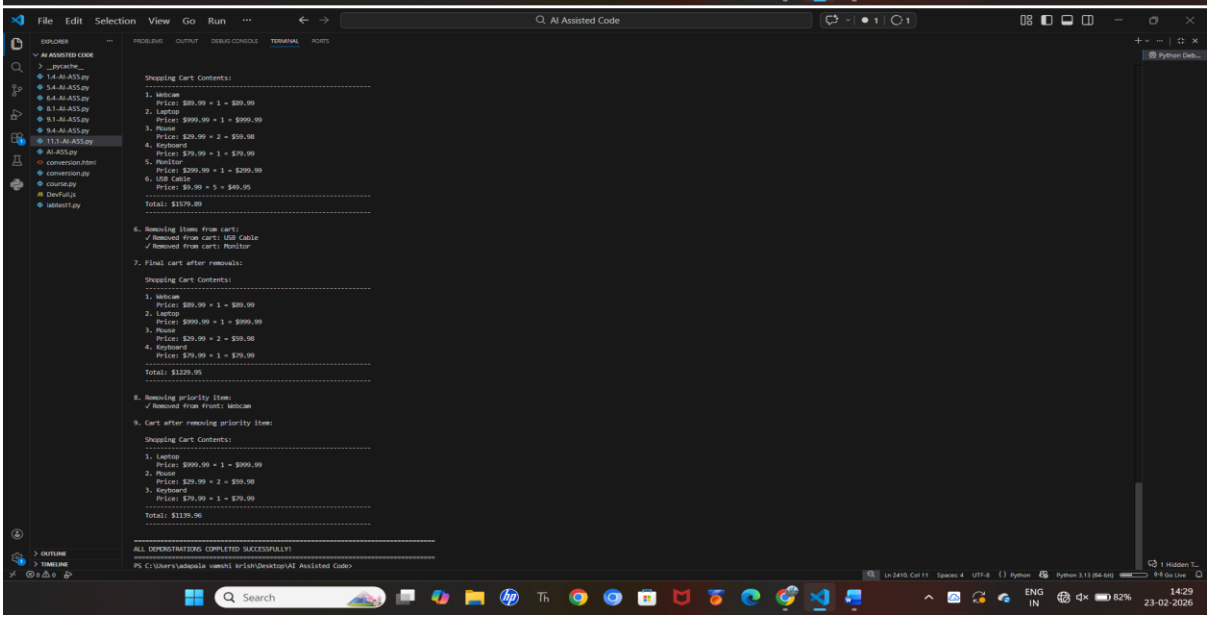
"Evaluate the Smart E-Commerce Platform features and choose the most appropriate data structure for each from Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, and Deque. Provide 2–3 sentence justifications for each choice. Generate a well-commented Python implementation for one selected feature."

Code:









Observation:

The AI successfully selected optimal data structures such as Queue for order processing, Priority Queue for ranking products, Hash Table for fast product lookup, and Graph for delivery routing. The implementation was efficient and clearly documented. This activity showed how understanding data structure behavior helps in designing scalable systems and how AI enhances productivity and code clarity.